

Hexadecimal floating literals for C++

Thomas Köppe <tkoepp@google.com>

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Impact](#)
4. [Proposed wording](#)
5. [An iostreams issue](#)
6. [Existing practice and implementation experience](#)

Revision history

P0245r0: Initial proposal.

Changes from P0245r0 to P0245r1:

- Change “significand part” to “significand”.
- Add example.
- Change first and second sentence of paragraph 1 to call out the prefix better.

Summary

There is a base-2 scientific notation for floating point values: `0x3.ABCp-10`. The significand is given in hexadecimal, and the exponent is given in decimal and interpreted with respect to base 2. This notation is convenient for certain uses. For example, the smallest normal IEEE-754 single precision value is readily written as `0x1.0p-126`. More generally, this notation can be used when an exact object representation is desired (and the floating point implementation uses a base-2 or base-2ⁿ representation).

The standard library outputs this representation via the `hexfloat` manipulator for ostreams, and via the `a/A` conversion specifiers in `printf`. Similarly, formatted input works because `strtod` recognises this representation (and the semantics of formatted istream input operations and `scanf` are defined in terms of `strtod`, but see below for a detail). We should also allow this representation as a literal in the core language.

Note that the C language which is referenced by the C++ standard also allows this literal syntax.

Impact

This is a core language extension that adds a new numeric literal syntax. The proposed syntax is ill-formed in the current working draft. Note that non-standard user-defined literals must start with an underscore, and the standard library does not define a UDL operator named `p` or `P`.

Proposed wording

☐hide deleted text

In section [lex.fcon] (2.13.4 in the working draft), change the grammar as follows (cf. C11 6.4.4.2/1).

floating-literal:

[decimal-floating-literal](#)

[hexadecimal-floating-literal](#)

decimal-floating-literal:

fractional-constant *exponent-part*_{opt} *floating-suffix*_{opt}

digit-sequence *exponent-part* *floating-suffix*_{opt}

hexadecimal-floating-literal:

hexadecimal-prefix *hexadecimal-fractional-constant* *binary-exponent-part* *floating-suffix*_{opt}

hexadecimal-prefix *hexadecimal-digit-sequence* *binary-exponent-part* *floating-suffix*_{opt}

fractional-constant:

*digit-sequence*_{opt} . *digit-sequence*

digit-sequence .

hexadecimal-fraction-constant:

*hexadecimal-digit-sequence*_{opt} . *hexadecimal-digit-sequence*

hexadecimal-digit-sequence .

exponent-part:

e *sign*_{opt} *digit-sequence*

E *sign*_{opt} *digit-sequence*

binary-exponent-part:

p *sign*_{opt} *digit-sequence*

P *sign*_{opt} *digit-sequence*

sign: one of

+ -

digit-sequence:

digit

digit-sequence ' _{opt} *digit*

hexadecimal-digit-sequence:

hexadecimal-digit

hexadecimal-digit-sequence ' _{opt} *hexadecimal-digit*

hexadecimal-prefix: one of

0x 0X

floating-suffix: one of

f l F L

Change the subsequent paragraph 1 in the following way:

A floating literal consists of an optional prefix specifying a base, an integer part, a ~~decimal~~radix point, a fraction part, an ~~e or E~~, e, E, p or P, an optionally signed integer exponent, and an optional type suffix. The integer and fraction parts both consist of a sequence of decimal (base ten) digits ~~if there is no prefix, or hexadecimal (base sixteen) digits if the prefix is 0x or 0X~~. The literal is a decimal floating literal in the former case and a hexadecimal floating literal in the latter case. Optional separating single quotes in a *digit-sequence* or hexadecimal-digit-sequence are ignored when determining its value. [Example. The literals 1.602'176'565e-19 and 1.602176565e-19 have the same value. – end example] Either the integer part or the fraction part (not both) can be omitted, ~~either~~, Either the ~~decimal~~radix point or the letter ~~e (or E)~~or E and the exponent (not both) can be omitted from a decimal floating literal. The radix point (but not the exponent) can be omitted from a hexadecimal floating literal. The integer part, the optional ~~decimal~~radix point and the optional fraction part form the ~~significant part~~significant of the floating literal. ~~The~~In a decimal floating literal, the exponent, if present, indicates the power of 10 by which the ~~significant part~~significant is to be scaled. In a hexadecimal floating literal, the exponent indicates the power of 2 by which the significand is to be scaled. [Example. The literals 49.625 and 0xC.68p+2 have the same value. – end example] If the scaled value is in the range of representable values for its type, the result is the scaled value if representable, else the larger or smaller representable value nearest the scaled value, chosen in an implementation-defined manner. The type of a floating literal is `double` unless explicitly specified by a

suffix. The suffixes `f` and `F` specify `float`, the suffixes `l` and `L` specify `long double`. If the scaled value is not in the range of representable values for its type, the program is ill-formed.

Drafting note. We changed “significant part” to “significand” and “decimal point” to “radix point”.

Also change the grammar in section [lex.ext] (2.13.8):

```
user-defined-floating-literal:  
  fractional-constant exponent-partopt ud-suffix  
  digit-sequence exponent-part ud-suffix  
hexadecimal-prefix hexadecimal-fractional-constant binary-exponent-part ud-suffix  
hexadecimal-prefix hexadecimal-digit-sequence binary-exponent-part ud-suffix
```

An iostreams issue

Although the library rules in [facet.num.get.members] say that formatted floating point input uses the conversion rules of `strtod` etc., the section also restricts the set of characters permissible for floating point parsing to “`0123456789abcdefxABCDEFX+-`”, which is missing the values `p` and `P`. Should the present proposal succeed, we plan to either create a new library proposal, or revisit [LWG issue 2381](#) to add support for parsing hexadecimal floating point literals to the standard library.

Existing practice and implementation experience

Hexadecimal float literals are already supported as a vendor extension by several compilers: Clang, EDG, Embarcadero, GCC. Formatted input via iostreams is supported by libc++.